

PYTHON

Dr. **Bülent Çobanoğlu**

cobanoglubulent[at]gmail.com

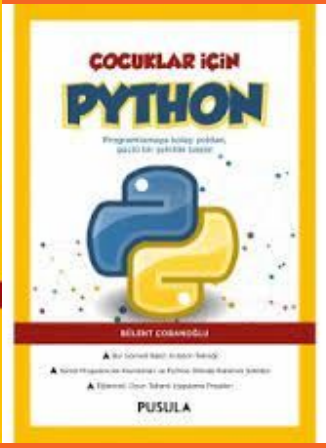
Ben kimim?

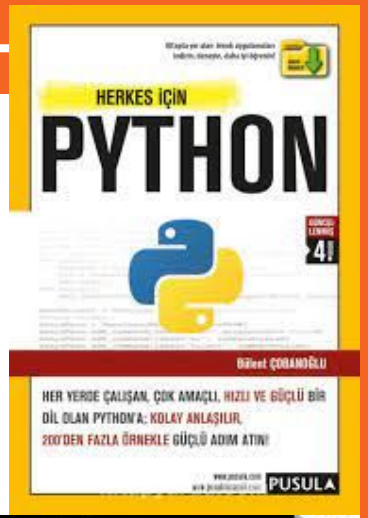
Kaynaklar;

- Eğitmen notları, https://github.com/bcobanoglu
- Python resmi sitesi

https://docs.python.org/3/









İçindekiler

- Set
- Frozenset,
- Dict,
- Nested Dict
- zip() fonksiyonu
- pandas ile sözlükten tablo oluşturma
- pandas ile listelerden tablo oluşturma
- Hata Yönetimi (try, except, else, finally)
- def ile fonksiyon tanımlama
- Lambda fonksiyonlar
- Koleksiyonlar üzerinde çalışan fonksiyonlar(map, filter, reduce,..)
- liste üreteçleri
- sözlük üreteçleri

dict metotları

`clear``()` :Sözlüğü boşaltır.

`copy``():` Sözlüğün bir kopyasını oluşturur.

`pop``(`*`key`*`[, `*`default`*`]):` `Sözlükte` *key* varsa onu siler ve değerini döndürür, yoksa *default* mesajı döndürür. Eğer *default* belirtilmez ise ve key'de sözlükte yoksa bir `KeyError` fırlatır.

`popitem``():` Sözlüğün en sonundaki `(key, value)` değer çiftini (elemanı) siler. Sözlük boşsa `KeyError` fırlatır.

`get``(`*`key`*`[, `*`default`*`]):` Aranan *key* sözlükte ise değerini yoksa *default değeri döndürür.* Default belirtilmezse, varsayılan değer None olur. Hiçbir zaman KeyError fırşatmaz. Sözlüğü değiştirmez

`setdefault``(`*`key`*`[, `*`default`*`]):` get ile aynı işleve sahiptir. Farkı aranan yoksa sözlüğe ekler.

`update``([`*`other`*`]):` Sözlüğü günceller. Örnek; `d.update(red=1, blue=2)`.

---

https://docs.python.org/3/library/stdtypes.html#dict

Sözlük Metotları



- **Task** 👇
  - Bir ingilizce-türkçe sözlük oluşturup, arama, güncelleme ve silme işlemlerini gerçekleştiriniz.

Task 👇

- Bir ingilizce-türkçe sözlük oluşturup, arama ve silme işlemlerini gerçekleştiriniz.



```
trEn = {"pazartesi": "monday", "salı": "tuesday", "çarşamba": "wednesday",
"perşembe": "thursday","cuma": "friday", "cumartesi": "saturday","pazar": "sunday"}
ara = input("Türkçe gün adı.:")
print("İngilizcesi.:", end=" ")
print(trEn.get(ara, "Bu kelime sözlükte yoktur!"))
```

```
sil = input("Silmek istediğiniz Türkçe gün adı.:")
print("Silinen..: ", trEn.pop(sil,"Sözlükte yoktur"))
print(trEn.items())
```

Sözlükten çoklu değer silme

Yöntem-1

```
trEn = list(trEn.items())      ## listeye dönüştürüp
del trEn[4:7]                  ## dilimleme yapılabilir
print(dict(trEn))
```

Yöntem-2

```
del trEn['cuma'], trEn['cumartesi'], trEn['pazar']
print(trEn)
```

Yöntem-3

```
del_keys = ['cuma', 'cumartesi', 'pazar']
## Bir döngü içerisinde silinebilir.
for key in del_keys:
    del trEn[key]
print(trEn)
```

unpacking operatörleri: * , **

- Sözlüklerde **, diğer iterable nesnelerde * kullanılır.

```
>>> print(*[1], *[2], 3)
1 2 3
>>> dict(**{'x': 1}, y=2, **{'z': 3})
{'x': 1, 'y': 2, 'z': 3}
```

```
>>> *range(4), 4
(0, 1, 2, 3, 4)
>>> [*range(4), 4]
[0, 1, 2, 3, 4]
>>> {*range(4), 4}
{0, 1, 2, 3, 4}
>>> {'x': 1, **{'y': 2}}
{'x': 1, 'y': 2}
```

Sözlükten Tabloya Dönüşüm

Sözlük yapısı pandas modülüne ait **DataFrame()** metodu ile tabloya dönüştürülebilir;

```
tablo = pandas.DataFrame(personel)
```

```
1  personel = {'ad' : ["al", "be", "ci", "di"],            # ad sütunu verileri
2              'yas' : [34, 25, 44, 54],                   # yas sütunu verileri
3              'bolum': ["MUH", "CEO", "ARGE", "ARGE"] }   # bolum sütunu verileri
4
5  import pandas as pd
6  tablo = pd.DataFrame(personel)
7  print(tablo)
```

✓ 0.0s

```
   ad  yas bolum
0  al   34   MUH
1  be   25   CEO
2  ci   44  ARGE
3  di   54  ARGE
```

Sözlükten Tabloya Dönüşüm

Sözlük yapısı tabulate modülüne ait **tabulate()** metodu ile de tabloya dönüştürülebilir;

```
tablo = tabulate.tabulate(personel,headers='keys')
```

```
1  import tabulate as tb
2  tablo = tb.tabulate(personel, headers='keys')
3  print(tablo)
```

✓ 0.0s

```
ad      yas  bolum
----  -----  -------
al       34  MUH
be       25  CEO
ci       44  ARGE
di       54  ARGE
```

Listeden Tabloya Dönüşüm

Sözlük yapısı pandas modülüne ait **DataFrame()** metodu ile tabloya dönüştürülebilir;

```
tablo = pandas.DataFrame(personel)
```

```
1  personel = {'ad' : ["al", "be", "ci", "di"],            # ad sütunu verileri
2              'yas' : [34, 25, 44, 54],                   # yas sütunu verileri
3              'bolum': ["MUH", "CEO", "ARGE", "ARGE"] }   # bolum sütunu verileri
4
5  import pandas as pd
6  tablo = pd.DataFrame(personel)
7  print(tablo)
```

✓ 0.0s

```
   ad  yas bolum
0  al   34   MUH
1  be   25   CEO
2  ci   44  ARGE
3  di   54  ARGE
```

İç içe (Nested) Sözlük

```
rehber = {
        'aile':
        {'anne_tarafi':{
            'dayi':{'kemal':'5652625950', 'bilal':'5607226924'},
            'teyze':{'sultan':'5683402450', 'vesile':'5655594181'},
            },
        'baba_tarafi':{
            'amca':{'şükrü':'5642066353', 'turan':'5668424288'},
            'hala':{'ayşe':'5666656772', 'kıymet':'5832380957'},
        },
        #....
        }
    }
```

İç içe (Nested) Sözlük

- rehber sözlük yapısında ‘bilal dayının telefonuna nasıl erişebiliriz?’

```
rehber = {
        'aile':
        {'anne_tarafi':{
            'dayi':{'kemal':'5652625950', 'bilal':'5607226924'},
            'teyze':{'sultan':'5683402450', 'vesile':'5655594181'},
            },
        'baba_tarafi':{
            'amca':{'şükrü':'5642066353', 'turan':'5668424288'},
            'hala':{'ayşe':'5666656772', 'kıymet':'5832380957'},
        },
        #....
        }
    }
```

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

İç içe (Nested) Sözlük

- rehber sözlük yapısında ‘bilal dayının telefonuna nasıl erişebiliriz?’

```
rehber = {
        'aile':
        {'anne_tarafi':{
            'dayi':{'kemal':'5652625950', 'bilal':'5607226924'},
            'teyze':{'sultan':'5683402450', 'vesile':'5655594181'},
            },
        'baba_tarafi':{
            'amca':{'şükrü':'5642066353', 'turan':'5668424288'},
            'hala':{'ayşe':'5666656772', 'kıymet':'5832380957'},
        },
        #....
        }
    }
# bilal dayının telefonunu alalım
print(rehber['aile']['anne_tarafi']['dayi']['bilal'])
```

İç içe (Nested) Sözlük

- **Task** 👇

- Arkadaşlarınızın ve aile üyelerinizin ad ve soyadlarından oluşan bir (**yakinim**) adlı bir sözlük oluşturup, elemanlarına erişim sağlayalım.
- Sözlük, her bir üyenin ad ve mail adresinden oluşmalı ve grupların (arkadaş ve aile) her biri üç kişiden oluşmalıdır. Örneğin;

  arkadaş :

  arkadas1: (ad : Sude, mail : sude@gmail.com)

  aile : _____

İç içe (Nested) Sözlük

- **Q:** 👇

  ‘c’ nin değerine (30 verisine) nasıl erişiriz?

```
data = [
  'x',
  'y',
  {
      'foo': 11,
      'bar':
      {
          'a' : 10,
          'b' : 20,
          'c' : 30
      },
      'baz': 33
  },
  'z',
  'q',
]
```



Students, write your response!

İç içe (Nested) Sözlük

- **Q:** 👇

  ‘c’ nin değerine(30 verisine) nasıl erişiriz?

```
data[2]['bar']['c']
```

```
data = [
  'x',
  'y',
  {
      'foo': 11,
      'bar':
      {
          'a' : 10,
          'b' : 20,
          'c' : 30
      },
      'baz': 33
  },
  'z',
  'q',
]
```

Sözlüklerde min-max işlemleri

- Q: 👇

```
notes = { 'ali': 90, 'bera': 42, 'cem': 66, 'dede': 100,
'eda':88 }
```

Bu sözlükteki en yüksek ve düşük notların kime ait olduğunu nasıl buluruz?



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Sözlüklerde min-max işlemleri

- **Q:** 👇

```
nots = { 'ali': 90, 'bera': 42, 'cem': 66, 'dede': 100,
'eda':88 }
```

Bu sözlükteki en yüksek ve düşük notların kime ait olduğunu nasıl buluruz?

```
nots = { 'ali': 90, 'bera': 42, 'cem': 66, 'dede': 100, 'eda':88 }
enk = min(nots, key=nots.__getitem__)
enb = max(nots, key=nots.__getitem__)
print ("en düşük not: ", enk)
print ("en yüksek not: ", enb)
```

✓ 0.0s

```
en düşük not:  bera
en yüksek not:  dede
```

Küme (Set) Veri Yapısı

- **Küme (Set)**; **sırasız, tekil elemanlar** topluluğudur. Küme elemanları süslü `{}` parantezler arasına yazılır.
- Matematiksel küme işlemleri yanında basit ekleme ve silme yöntemleri vardır.

- `{}`
- `set()`

```
set1 = {'mavi', 'yeşil', 'sarı', 'siyah'}
colors = 'mavi', 'yeşil', 'sarı', 'siyah'
set2 = set(colors)
print(set1)
print(set2)
```

✓ 0.0s

```
{'sarı', 'siyah', 'yeşil', 'mavi'}
{'sarı', 'siyah', 'yeşil', 'mavi'}
```

Boş Küme

- **Boş küme** tanımlanırken **`{}` kullanılmaz,** **`set()`** deyimi kullanılır. **Örnek;**



```
bos_kume = set()
print(type(bos_kume))
```

✓ 0.0s

```
<class 'set'>
```



```
bos_kume = {}
print(type(bos_kume))
```

✓ 0.0s

```
<class 'dict'>
```

Küme - List Farkı

- Aşağıdaki programın çıktısı nedir?

```
set1 = {'mavi', 'yeşil', 'sarı', 'siyah', 'mavi'}
list1 = ['mavi', 'yeşil', 'sarı', 'siyah', 'mavi']


print(set1)
print(list1)
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Küme metotları

Küme Metotları	Açıklaması	Örnek Kullanım

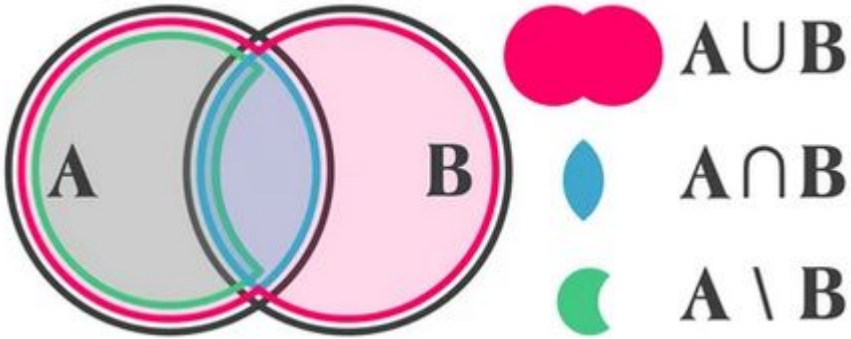
K.add(x)	Kümeye x elemanını ekler.	K=set()# K boş küme
		K.add('elma')
		print(K) # {'elma'}

K.copy()	Listelerdeki gibi kümenin bir kopyasını oluşturur.	B={ 1,7, 11, 3 } C=B.copy() print(C) #{1,7,11,3}
K.remove(x)	Kümedeki 'x' elemanını siler. Eleman yoksa hata mesajı verir.	B.remove(7) #7'i siler print (B) #{1, 11, 3}
K.discard(x)	Kümedeki 'x' elemanını siler. Eleman yoksa herhangi bir hata mesajı vermez.	B.discard(7) #7'i siler print (B) #{1, 11, 3}
K.pop()	Kümenin ilk elemanını siler.	B= { 1,7, 11, 13,7 } B.pop() print(B) #{11, 13, 7}
K.clear()	Kümedeki tüm elemanları siler. Boş küme oluşturur.	K= {1, 3, 5, 7} K.clear() print (K) #set()

- `A.isdisjoint(B)` : A ile B nin kesişimi boş ise T**rue** değilse F**alse** döndürür
- A.**update(**B) : Eşdeğeri **A |=B** dir, A yı B ile günceller

Küme işlemleri

- **`set()`** yapısı tüm matematiksel küme işlemlerini destekler.




```
-   : 'difference',
-=  : 'difference_update',

&   : 'intersection',
&=  : 'intersection_update',

<=  : 'issubset',
>=  : 'issuperset',

^   : 'symmetric_difference',
^=  : 'symmetric_difference_update',

|   : 'union'
```

Aşağıdaki programın çıktısı nedir?

```
A = set('mecidiye')
B = set('selimiye')
print(A.union(B))
print(A|B)

print(A.difference(B))
print(A-B)

print(A.intersection(B))
print(A&B)
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Küme işlemleri

- **Aşağıdaki programın çıktısı nedir?**

```
A = set('mecidiye')
B = set('selimiye')
print(A.union(B))
print(A|B)
```

```
{'y', 'e', 'c', 'd', 'l', 'i', 'm', 's'}
{'y', 'e', 'c', 'd', 'l', 'i', 'm', 's'}
```

```
print(A.difference(B))
print(A-B)
```

```
{'c', 'd'}
{'c', 'd'}
```

```
print(A.intersection(B))
print(A&B)
```

```
{'y', 'e', 'i', 'm'}
{'y', 'e', 'i', 'm'}
```

Sözlüklerde Küme İşlemleri

- iki sözlük küme birleştirme operatörü ile ‘A|B’ şeklinde birleştirilebilir ve ‘A |= B’ şeklinde güncellenebilir.

```
A = {'1':"Bir", '2':"İki", '3':"Üç", '4':"Dört"}
B = {1:"One", 2:"Two", '4':"Four", 5:"Five"}
A |= B
print (A)
```

✓ 0.0s

```
{'1': 'Bir', '2': 'İki', '3': 'Üç', '4': 'Four', 1: 'One', 2: 'Two', 5: 'Five'}
```

Küme işlemleri

- **Aşağıdaki listedeki kayıp elemanı nasıl bulabiliriz?**
- `nums = [2, 3, 1, 5, 6, 4, 9, 8, 10]`



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Küme işlemleri

- Aşağıdaki listedeki kayıp elemanı nasıl bulabiliriz?

```
nums = [2, 3, 1, 5, 6, 4, 9, 8, 10]

sorted_list = sorted(nums)
print("Sirali liste:", sorted_list)

missing_num = set(range(1, 11)) - set(nums) #difference
print("Kayıp Sayi:", missing_num.pop())
```

✓ 0.0s

```
Sirali liste: [1, 2, 3, 4, 5, 6, 8, 9, 10]
Kayıp Sayi: 7
```

frozenset()

- **frozenset()** küme elemanlarını **'değiştirilemez (immutable)'** yapar.

```
kume = {'mavi', 'yeşil', 'sarı', 'siyah'}
kume = frozenset(kume)
kume.add("kırmızı")
print(kume)
```

⊗ 0.0s

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
/home/bull/py_projects/d05_python_dicts_09.ipynb Cell 104 in ()
      1 kume = {'mavi', 'yeşil', 'sarı', 'siyah'}
      2 kume = frozenset(kume)
----> 3 kume.add("kırmızı")
      4 print(kume)

AttributeError: 'frozenset' object has no attribute 'add'
```

Mutable or Immutable ?

immutable tipler;

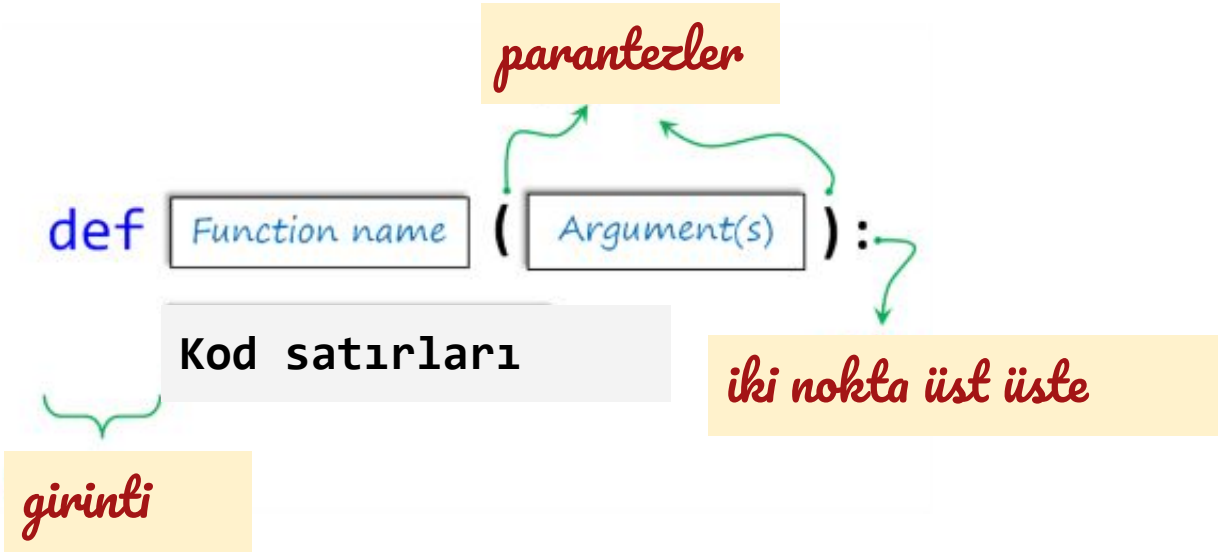
```
int, float, decimal, complex, bool, string, tuple, range, frozenset, bytes
```

mutable tipler;

```
list, dict, set, bytearray, user-defined classes
```

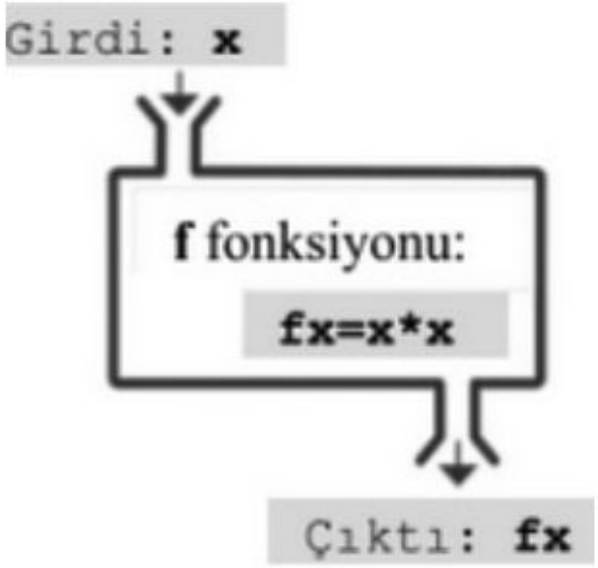
Kullanıcı tanımlı fonksiyonlar

- Kendi fonksiyonumuzu '**def**' deyimi ile tanımlayabiliriz;



Kullanıcı tanımlı fonksiyonlar

- Fonksiyon tanımlanırken belirtilen girdilere '**parametre'**, çağrılırken belirtilen girdilere ise '**argüman**' denir.




```
# fonksiyon tanımlama
def f(x):
    fx = x*x
    return fx


#fonksiyon çağırma
f(5)   # 25
```

Fonksiyon Tanımlama



- **Task**:👉 **carp()** isimli bir fonksiyon tanımlayalım ve aşağıdaki değerler ile test edelim?

```
carp(2,3)
carp(-3,2.5)
carp('Python',3)
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

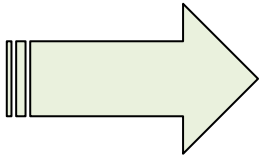
Fonksiyon Tanımlama



- **Task**:👉 **carp()** isimli bir fonksiyon tanımlayalım

```
#fonksiyon tanımlandı
def carp(a,b):
    print(a*b)

#fonksiyon çağrıldı
carp(2,3)
carp(-3,2.5)
carp('Python',3)
```



```
6
-7.5
PythonPythonPython
```

return

Komut örneği	Çıktı değeri	Açıklama
return 7	7	Bu örnekte fonksiyonun geri dönüş (çıktı) değeri sabit bir sayıdır
return x	x	Bu örnekte fonksiyonun çıktısı bir değişkendir.
return x+1	x+1	Bu örnekte fonksiyonun çıktısı bir değişkendir.
return f(x,y)	f()	Bu örnekte fonksiyonun geri dönüş değeri, başka bir fonksiyonun “f()” çıktısıdır.
return x,y	x ve y	Python dilinde return ile aynı anda birden fazla değer geri döndürülebilir.
return	None	Bu örnekte fonksiyonun bir çıktısı yoktur. return, sadece fonksiyonu sonlandırır.

return - print farkı:

- **Task**:👉 Bu iki fonksiyon arasındaki fark nedir?

```
def carp1(a,b):
    print(a*b)
```

```
def carp2(a,b):
    return(a*b)
```

return - print farkı:

- Eğer bir fonksiyon bir değer üretmiyorsa **None** değerini döndürür.

```
def carp1(a,b):
    print(a*b)

print(carp1(3,4))
print(type(carp1(3,4)))
```

✓ 0.0s

```
12
None
12
<class 'NoneType'>
```

```
def carp2(a,b):
    return(a*b)

print(carp2(3,4))
print(type(carp2(3,4)))
```

✓ 0.0s

```
12
<class 'int'>
```

pass deyimi

- **İçi boş** fonksiyon, sınıf, döngü tanımlanmasında kullanılır;

```
def turko_vac():
    pass
```

Konumsal(Positional) argüman



- Argümanlar sırasıyla fonksiyonun parametrelerine atanır. Sıra önemlidir. Buna göre **çıktısı nedir?**

```
def mesaj(x,y,z):
  print(f"{y} {x} {z}")


#Ana program
a="I"; b = "love"; c = "you"
mesaj(1,2,3)
mesaj(a,b,c)
mesaj("top", "ali", "at")
mesaj(b,a,c)
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Konumsal(Positional) argüman



- Positional argümanlar soldan sağa doğru sırası ile aktarılır.

```
def mesaj(x,y,z):
  print(f"{y} {x} {z}")

#Ana program
a="I"; b = "love"; c = "you"
mesaj(1,2,3)
mesaj(a,b,c)
mesaj("top", "ali", "at")
mesaj(b,a,c)
```

```
2 1 3
love I you
ali top at
I love you
```

Keyword (kwargs) argüman



- Keyword argümanlar, açıkça parametre adlarıyla (sözlük yapısındaki anahtar-değeri şeklinde) eşleştirilir. Sıralama önemli değildir.

```
def mesaj(x="love",y="i",z="you"):
    print(f"{y} {x} {z}")


#fonksiyon çağrılıyor
mesaj()
mesaj(z="you", x="i", y="love")
mesaj(x="top", y="ali", z="at")
mesaj("i", "ben")
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Keyword argüman



- Positional argümanlar soldan sağa doğru sırası ile aktarılır. Buna göre **çıktısı nedir?**

```
def mesaj(x="love",y="i",z="you"):
    print(f"{y} {x} {z}")


mesaj()
mesaj(z="you", x="i", y="love")
mesaj(x="top", y="ali", z="at")
mesaj("i", "ben")
```

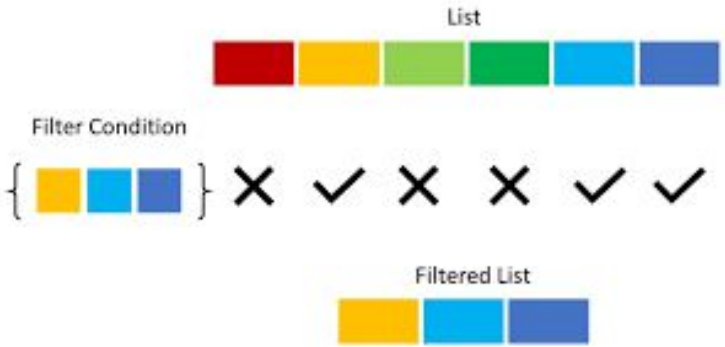
```
i love you
love i you
ali top at
ben i you
```

Liste elemanları üzerinde işlem yapan fonksiyonlar

- Liste içerisindeki elemanları bir işleme tabi tutarak yeni listeler üretmek veya tek bir değer üretmek (toplama,çıkarma, çarpma gibi) için **reduce, filter, map veya zip** gibi fonksiyonlar kullanılır.

filter() fonksiyonu

- Bir grup iterable veriyi belirli bir kritere (veya fonksiyona) göre filtrelemek için kullanılır;



`Not. itertools.filterfalse()` ise tam tersi bir işleve sahiptir.

filter (*function, iterable*)



- Çıktısı nedir?

```
import keyword
vars = ["value", "file#", "var", "not", "444Ali", "and", "ad soyad"]
print(list(filter(str.isidentifier, vars)))
print(list(filter(keyword.iskeyword, vars)))
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

filter (*function, iterable*)



- Çıktısı nedir?

```
import keyword
vars = ["value", "file#", "var", "not", "444Ali", "and", "ad soyad"]
print(list(filter(str.isidentifier, vars)))
print(list(filter(keyword.iskeyword, vars)))
```

```
['value', 'var', 'not', 'and']
['not', 'and']
```

filter (*function, iterable*)



- Çıktısı nedir?

```
lis = [1,3,5, None, False, 0, '0.0', 'Ali', [], 'Koç']
filtered_lis = list(filter(None, lis))
print(filtered_lis)
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

filter (*function, iterable*)



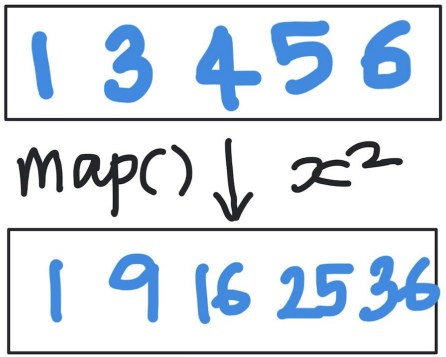
- Çıktısı nedir?

```
lis = [1,3,5, None, False, 0, '0.0', 'Ali', [], 'Koç']
filtered_lis = list(filter(None, lis))
print(filtered_lis)
```

```
[1, 3, 5, '0.0', 'Ali', 'Koç']
```

map() fonksiyonu

- Bir grup veriyi (liste, demet) belli bir kriter (veya fonksiyon) ile eşleştirmek için **map()** fonksiyonu kullanılır;



Not. map() birebir eşleştirme yapar, girdi sayısı kadar çıktı verir.

map (*function, iterable*)



- Çıktısı nedir?

```
lis = [(1,3), 'None', 'Ali', [1,2,3,4,5]]
map_lis = list(map(len, lis))
print(map_lis)
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

map (*function, iterable*)



- Çıktısı nedir?

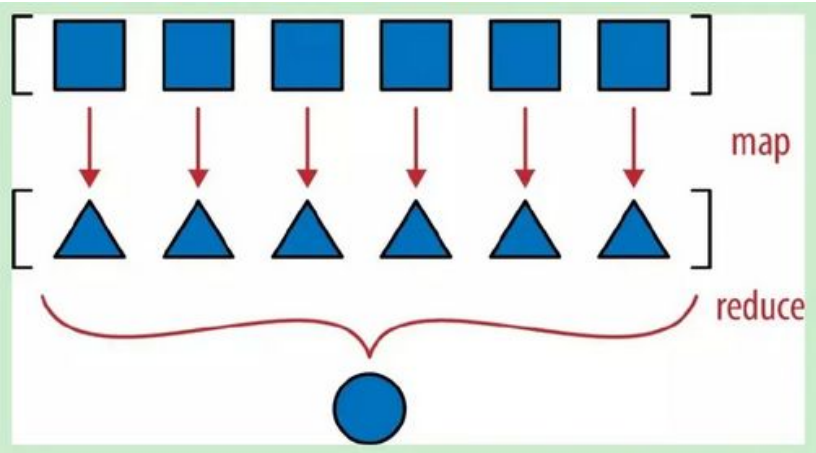
```
lis = [(1,3), 'None', 'Ali', [1,2,3,4,5]]
map_lis = list(map(len, lis))
print(map_lis)
```

lis ile map_lis'in eleman sayıları eşit...

```
[2, 4, 3, 5]
```

reduce() fonksiyonu

- Bir grup veriden (listeden) **tek bir sonuç üretmek** için reduce(azaltma/indirgeme) fonksiyonu kullanılır;
- functools.reduce(function, iterable)



Not. reduce() tek bir anlamlı değer üretir.

reduce ()



- Çıktısı nedir?

```
from functools import reduce
def carp(a,b):
    return a*b

liste = [1,2,3,4,5]
print(reduce(carp, liste))
print(reduce(carp, liste, 100))
```

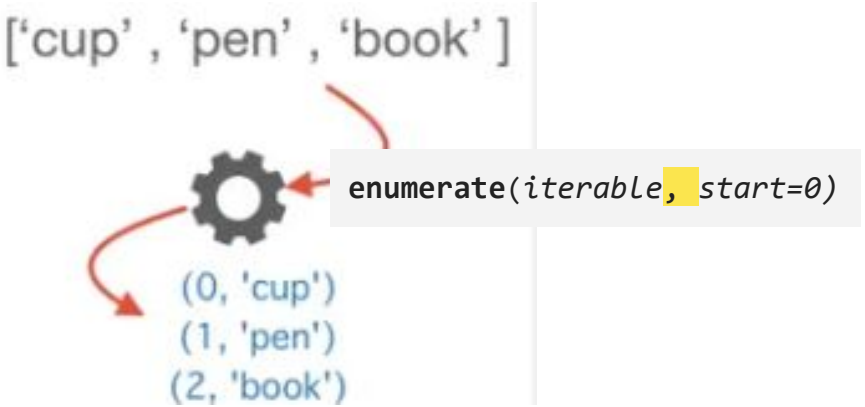


Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

enumerate() fonksiyonu

- enumerate() fonksiyonu, kelime manasına uygun olarak verileri **numaralandırma**k amacıyla kullanılır;



enumerate() fonksiyonu

```
>>> list(enumerate('Python'))
[(0, 'P'), (1, 'y'), (2, 't'), (3, 'h'), (4, 'o'), (5, 'n')]
```

#'start=no' ile varsayılan indis numarası değiştirilebilir.

```
>>> list(enumerate('Python', start=1))
[(1, 'P'), (2, 'y'), (3, 't'), (4, 'h'), (5, 'o'), (6, 'n')]
```

enumerate (*iterable, start=0*)



- **Task**:👉 Haftanın günlerinin rakamsal karşılığını veren programı nasıl kodlarız?

```
1.gün: Pazartesi
2.gün: Salı
3.gün: Çarşamba
4.gün: Perşembe
5.gün: Cuma
6.gün: Cumartesi
7.gün: Pazar
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

enumerate (*iterable, start=0*)



- **Task**:👉 Haftanın günlerinin rakamsal karşılığını veren programı nasıl kodlarız?

```
gun= ['Pazartesi', 'Salı', 'Çarşamba', 'Perşembe', 'Cuma', 'Cumartesi', 'Pazar']

for i, deger in enumerate(gun, start=1):
    print(str(i) + ".gün: " + deger)
```

✓ 0.0s

```
1.gün: Pazartesi
2.gün: Salı
3.gün: Çarşamba
4.gün: Perşembe
5.gün: Cuma
6.gün: Cumartesi
7.gün: Pazar
```

enumerate (*iterable, start=0*)



- **Task**:👉 Kursiyerlere otomatik numara veren programı kodlayınız?
- `names = ["Ali", "Seda", "Nihal", "Pelin", "Cuma"]`

```
10: Ali
11: Seda
12: Nihal
13: Pelin
14: Cuma
```



enumerate (*iterable, start=0*)



```
names = ["Ali", "Seda", "Nihal", "Pelin", "Cuma"]

for i, ad in enumerate(names, start=10):

    print(i, ad)
```

```
10: Ali
11: Seda
12: Nihal
13: Pelin
14: Cuma
```

zip() ile iki farklı listeyi sözlüğe fönüştürme

- İki farklı listeyi biri anahtar (key), diğeri değeri (value) olacak şekilde sözlük yapısına dönüştürmek için zip() fonksiyonu kullanılır.

```
ad = ['one','two','three','four','five']
no = [1, 2, 3, 4, 5 ]
szlk = dict(zip (ad,no))
print ("Sözlük: ", szlk)
```

✓ 0.0s

```
Sözlük:  {'one': 1, 'two': 2, 'three': 3, 'four': 4, 'five': 5}
```

Anonim fonksiyonlar: Lambda

- Tek satırlık kısa fonksiyonlar tanımlamak için Lambda ifadeleri kullanılır.
- Lambda ifadeleri **anonim (isimsiz)** fonksiyonlardır.
- Kullanım şekli;

```
1  def karesi(a):
2      return a**2
3
4  print(karesi(2))
```

eşdeğeri

```
1  karesi = lambda a : a**2
2
3  print(karesi(2))
```

Anonim fonksiyonlar: Lambda

- lambda fonksiyonlarında **return kullanılmaz.**
- parantez içerisinde doğrudan **argümanlar** belirtilebilir;
- `(lambda arguments : expression)(arguments)`

```
print((lambda a:a**2)(2))
```

- Birden fazla argüman alabilir;

```
print((lambda a,b:a+b)(3,4))  #7
```

Anonim fonksiyonlar: Lambda

- Çıktısı nedir?

```
for i in [3, 5, 4, 1, 8, 0]:
    print(i,':',(lambda i: "tek" if i%2 else "çift")(i))
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

map - lambda (inline func.)

- Çıktısı nedir?

```
liste = map(lambda a: a**2, range(1,7))
print(list(liste))
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

filter - lambda (inline func.)

- Çıktısı nedir?

```
liste = filter(lambda a: a%2==0, range(1,7))
print(list(liste))
```

Students choose an option

Pear Deck Interactive Slide
Do not remove this bar

filter-lambda (inline func.)

- Asgari ücretten yüksek maaş alanların sayısını veren programı filter(), lambda, len() fonksiyonlarını kullanarak tamamlayınız?

```
maas = [20_000, 24_000, 10_000, 46_000, 8_000, 9_600]
asgari_ucret= 10_008
#...
#....
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

map - filter (inline func.)

- Asgari ücretten yüksek maaş alanların sayısını veren programı filter(), lambda, len() fonksiyonlarını kullanarak tamamlayınız?

```
1  maas = [20_000, 24_000, 10_000, 46_000, 8_000, 9_600]
2  asgari_ucret= 10_008
3  yuksek_maas = list(filter(lambda m : m>asgari_ucret, maas))
4  print(len(yuksek_maas))
```

✓ 0.0s

```
3
```

Liste üreteçleri (List Comprehensions)

- Normal form:

```
liste = []
for i in range(10):
    liste.append(i)
```

- Kısaltılmış form:

```
liste = [i for i in range(10)]
```

Liste üreteçleri (List Comprehensions)

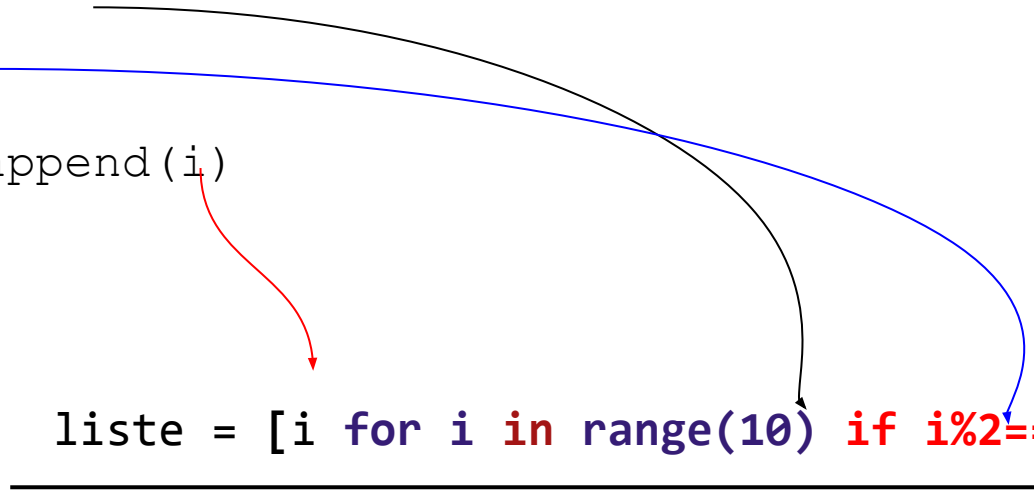
- Normal form:

```
liste = []

for i in range(10):
    if i%2==0:
        liste.append(i)
```

- Kısaltılmış form:

```
liste = [i for i in range(10) if i%2==0]
```



Liste üreteçleri (List Comprehensions)

- Normal form:

```
liste = []
for i in range(10):
    if i%2==0:
        liste.append(i)
    else:
        liste.append(i**2)
```

- Kısaltılmış form:

```
liste = [i if i%2==0 else i**2 for i in range(10)]
```

⏩ Çıktısı nedir?

Liste üreteçleri:

```
[i for i in range(100, 0,-10)]                    #out: ?
```

⏩ Çıktısı nedir?

Liste üreteçleri:

```
text = "Hatalar insanlar içindir"

[i for i in text if not i in 'aeiouıüö']    #out: ?
```

Küme Üreteçleri (Set comprehensions)

- **Aşağıdaki programın çıktısı nedir?**

```
>>> a = {x for x in 'abracadabra' if x not in 'abc'}
>>> a
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Küme Üreteçleri (Set comprehensions)

- **Aşağıdaki programın çıktısı nedir?**

```
>>> a = {x for x in 'abracadabra' if x not in 'abc'}
>>> a
{'r', 'd'}
```

Sözlük Üreteçleri (dict comprehensions)

- **Aşağıdaki programın çıktısı nedir?**

```
sozluk = {i : chr(65+i) for i in range(8)}
print(sozluk)
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Sözlük Üreteçleri (dict comprehensions)

- **Aşağıdaki programın çıktısı nedir?**

```
sozluk = {i : chr(65+i) for i in range(8)}
print(sozluk)
```

✓ 0.0s

```
{0: 'A', 1: 'B', 2: 'C', 3: 'D', 4: 'E', 5: 'F', 6: 'G', 7: 'H'}
```

Hatalar (Errors) ve Yakalama

Errors(Hatalar)



- 👉Şimdiye kadar en sık aldığınız hata mesajları nelerdi?

TypeError

NameError

SyntaxError

IndexError

ValueError

IndentationError

FileNotFoundError

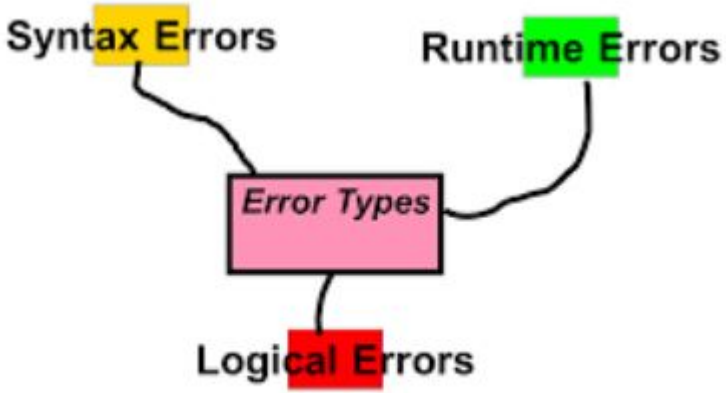


Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

SyntaxError - *Runtime* Errors

- 👉Bu hata türleri arasında ne gibi bir fark olabilir?



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Errors and Exceptions



- 👉Sözdizimi hatası (syntax errors)

```
>>> while True print('Hello world')
  File "<stdin>", line 1
    while True print('Hello world')
                    ^
SyntaxError: invalid syntax
```

- 👉Çalışma zamanı hataları (exceptions)

```
>>> 10 * (1/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
>>> 4 + spam*3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'spam' is not defined
>>> '2' + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int")
```

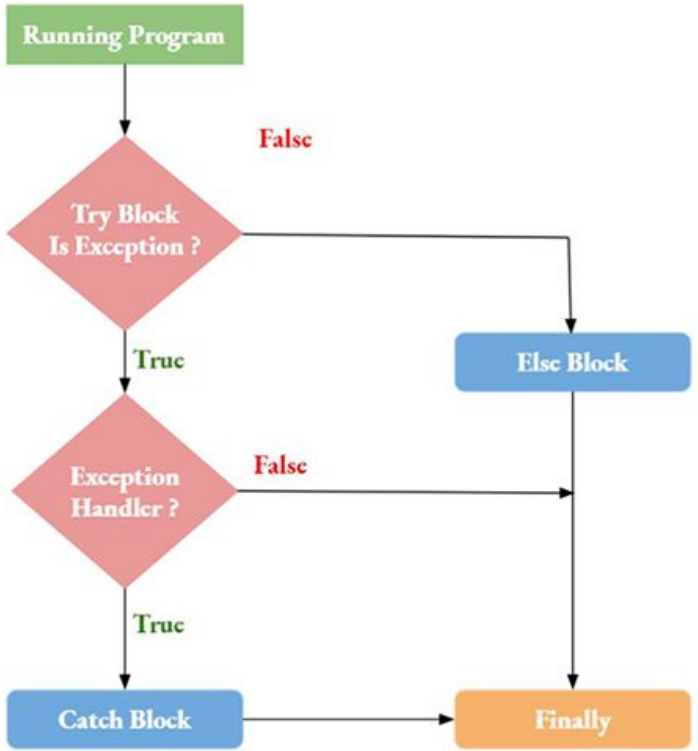
Hata Yakalama (Exception Handling)

- Python dilinde hata yakalama işlemleri dört anahtar sözcük ile yönetilir; 👉
- **try,**
- **except,**
- **else**
- ve **finally.**



Hata Yakalama (Exception Handling)

- Python dilinde hata yakalama işlemleri dört anahtar sözcük ile yönetilir; 👉
- **try,**
- **except,**
- **else**
- ve **finally.**



Hata Yakalama (Exception Handling)

- Python dilinde hata yakalama işlemleri dört anahtar sözcük ile yönetilir; 👉 **try, except, else** ve **finally.**

```python
def divide(x, y):
    try:
        result = x / y
    except ZeroDivisionError:
        print("division by zero!")
    else:
        print("result is", result)
    finally:
        print("executing finally clause")

divide(2, 1)
# hem sonucu : 2.0 yazar
# hemde finally bloğu çalışır
divide(2, 0)
# hem division by zero!
# hemde finally bloğu çalışır
```

Çalışma



- 👉Aşağıdaki programın çıktısı nedir?

```
def test():
    try:
        1 + 0
    except:
        return 0
    else:
        return 1
    finally:
        print("The end")
print(test())
```

a

```
The end
1
```

b

```
1
The end
```

c

```
The end
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Hata Yakalama (Exception Handling)

- 👉 **try, except** Hata yakalama bloğu bölünemez;



```
n = 5
try:
    print(n,end=' ')
    print('bölme beni!')

except:
    print('!')
```



```
n = 5
try:
    print(n,end=' ')
print('bölme beni!')

except:
    print('!')
```

Hata Yakalama (Exception Handling)

- 👉 **try** tek başına kullanılmaz!
- Ya '**except**' ya da '**finally**' bloğu olmak zorunda



```
n = 5
try:
    print(n,end=' ')
    print('bölme beni!')

except:
    print('!')
```

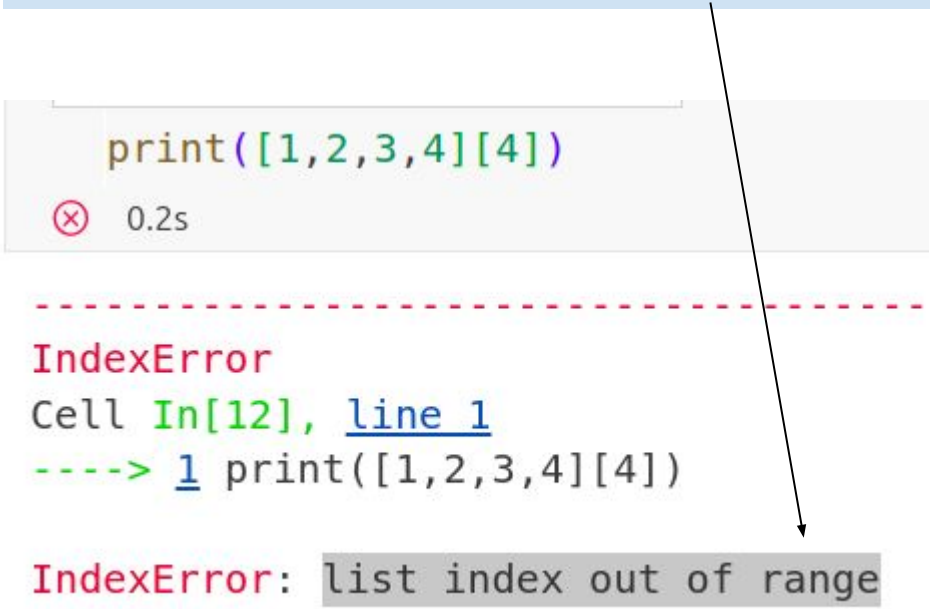


```
n = 5
try:
    print(n,end=' ')
print('!')
```

⊗ 0.0s

```
  Cell In[4], line 4
    print('!')
    ^
SyntaxError: expected 'except' or 'finally' block
```

associated value



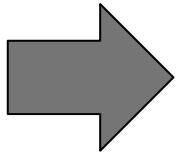
associated value

```
print([1,2,3,4][4])
```

⊗ 0.2s

```
---------------------------------------
IndexError
Cell In[12], line 1
----> 1 print([1,2,3,4][4])

IndexError: list index out of range
```



```
try:
    print([1,2,3,4][4])
except Exception as ex:
    print(ex)
```

✓ 0.0s

```
list index out of range
```